SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

BALA NAGA JASWANTH
192424300C

Assignment_1

← Back

QUESTIONS

Q1
Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System
50 marks

0/1 answered

Q1 Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

50 marks

Programming in Java
ASSIGNMENT 1
CO1: Java fundamentals and object-oriented programming - data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation
Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks - 50)
Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____
Mandatory Execution and GitHub Submission
• Every student must individually execute every task in this assignment, including all “Complete the Code” programs.
• Create a GitHub repository named CSA09-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
• Keep one folder for each task (for example: Task01,

Task02 ... or Q1, Q2 ...), containing the Java source code, input files and output screenshots.
• Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:

How to Answer the Additional Questions
Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Campus Smart-Parking System
The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.
When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release

the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.
The facility may contain:
1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots
You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.
Design Challenge:
The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.
Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks
Bloom's Level: K4 – Analyse
Analyse the parking problem and identify the objects that a Java program must model.
Discuss:
• The inputs the system receives and the outputs it must produce
• The nouns in the problem that become candidate classes, and the verbs that become methods
• The responsibilities of each candidate class
• The business rules every valid operation must satisfy (for example: a slot cannot hold two vehicles; a ticket number is never reused)
• Any assumptions you make about the facility

You are expected to justify why each candidate class is needed, not merely list names.
Alternative Concept and Complexity Analysis
In your design each vehicle is an object. If the vehicles were instead stored as plain Strings in

parallel arrays (regNo[], type[], entryTime[]), what would the solution look like? Compare both designs for finding a vehicle by registration number and for adding a new vehicle attribute. Analyse the time complexity of the search and the space used per vehicle in each design.
Evidence Expected
A clear problem analysis with candidate classes, their responsibilities, business rules and stated assumptions.

TASK 2 – Choose the Data Representation 3 Marks
Bloom's Level: K4 – Analyse
Decide how the data of the system will be represented using Java data types and arrays.
Discuss:
• The data type chosen for each field (slot number, vehicle number, entry time, duration, fee) and why
• How the floors and slots will be represented (one-dimensional, two-dimensional or jagged array) and why
• Where type conversion or casting is required (for example: converting minutes to hours for billing) and the risk of losing precision
• How you will read input (Scanner or command-line arguments)

Show at least one example where integer division or an incorrect cast would produce a wrong fee, and how your design avoids it.
Alternative Concept and Complexity Analysis
Instead of a jagged array String[floor][slot], suppose the slots are stored in a single one-dimensional array using index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. What would change in your solution when floors have different numbers of slots? Analyse the time to locate a slot and the space required in each representation.
Evidence Expected

Justified choice of data types and array structure, with an identified casting risk and its resolution.

TASK 3 – Design the Control Logic 4 Marks
Bloom's Level: K4 – Analyse
Design the decision and looping logic for the two core operations: allocating a free slot and computing the parking fee.
Discuss:
• How the program searches the floors and slots for a suitable free slot, and when the search stops
• How the fee slabs (for example: first hour, each additional hour, daily maximum) are expressed using if-else or switch
• How break, continue and return are used to avoid unnecessary work
• What happens when no suitable slot is free

Write pseudocode for both operations following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing Java code.
Complete the Code
Complete the class below and execute it. Do not change the main() method.

```
public class ParkingFacility {
// slots[floor][slot] holds a vehicle number, or null if the slot is free
private final String[][] slots;

public ParkingFacility(int[] slotsPerFloor) {
slots = new String[slotsPerFloor.length][];
for (int f = 0; f < slotsPerFloor.length; f++) {
slots[f] = new String[slotsPerFloor[f]];
}
}

/** Returns {floor, slot} of the first free slot, or null if it is full. */
public int[] allocate(String vehicleNo) {
// TODO 1: Search floor by floor and slot by slot; place vehicleNo in the first
// free slot and return {floor, slot}
```

```
return null;
}

/** Frees a slot. Returns false if the position is
invalid or already free. */
public boolean release(int floor, int slot) {
// TODO 2: Validate floor and slot, free the slot and
return true; otherwise
// return false

}

public static void main(String[] args) {
// floor 0 has 2 slots, floor 1 has 1 slot
ParkingFacility p = new ParkingFacility(new int[]{2,
1});
System.out.println(java.util.Arrays.toString(p.allocate
("TN01AB1111")));
System.out.println(java.util.Arrays.toString(p.allocate
("TN02CD2222")));
System.out.println(java.util.Arrays.toString(p.allocate
("TN03EF3333")));
System.out.println(java.util.Arrays.toString(p.allocate
("TN04GH4444")));
System.out.println(p.release(0, 1) + " " + p.release(0,
1) + " "
+ p.release(5, 0));
System.out.println(java.util.Arrays.toString(p.allocate
("TN04GH4444")));
}
}
```

Expected output:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Alternative Concept and Complexity Analysis
Your allocation scans every floor and slot until a free one is found. If, instead, you maintained a count of free slots per floor and a stack of free slot numbers, what would the allocation and release logic become? Analyse the time complexity of allocate() and release() and the extra space needed in both approaches for F floors and S slots per floor.
Evidence Expected
Complete pseudocode for slot allocation and fee computation with correct decision and loop logic.

TASK 4 – Design Classes, Objects and Constructors
4 Marks
Bloom's Level: K4 – Analyse
Design the classes identified in Task 1 and decide how their objects will be created.
Discuss:
• The fields and methods of each class
• Default, parameterised and overloaded constructors, and when each is used
• Use of the this keyword and constructor chaining to avoid repeated code
• Whether a copy constructor is needed anywhere (for example: re-issuing a duplicate ticket) and why
• How objects are stored in memory (reference variables, stack and heap)

Alternative Concept and Complexity Analysis
You used overloaded constructors and constructor chaining. If objects were instead created with a default constructor followed by setter calls, or through static factory methods, what would the solution be? Compare how each approach guarantees a valid object, and analyse the time and space cost of creating n ParkingTicket objects in each approach.
Evidence Expected
Class definitions with justified constructors, use of this and constructor chaining, and an explanation of object creation.

TASK 5 – Apply static Members and the Object Life Cycle 3 Marks
Bloom's Level: K4 – Analyse
Identify the data and behaviour that belong to the class as a whole rather than to individual objects.
Discuss:
• Where static variables are needed (for example: a ticket counter, total occupancy) and why an instance variable would be wrong
• Where static methods or a static block are appropriate
• What happens to a ParkingTicket object after the vehicle leaves, and when it becomes eligible for garbage collection
• How objects are passed to and returned from methods in your design

Alternative Concept and Complexity Analysis
The ticket counter is a static variable. If it were made an instance variable of ParkingFacility, what would happen when the university runs two facilities (north and south gates)? Propose the solution for each case and analyse the space used by the counter for k facility objects and n tickets.
Evidence Expected
Correct and justified use of static members with an explanation of object passing and the object life cycle.

TASK 6 – Design the Inheritance Hierarchy 4 Marks
Bloom's Level: K4 – Analyse
Design an inheritance hierarchy for the vehicles and, if appropriate, for the users (student, staff, visitor).
Discuss:
• The superclass and subclasses, and the type of inheritance used (single, multilevel or hierarchical)
• What is inherited and what is added in each subclass
• How super is used for fields, methods and constructors

• The order in which constructors execute when an ElectricCar object is created, shown by a trace

Draw the hierarchy as a UML class diagram.
Alternative Concept and Complexity Analysis
You used inheritance (Car is-a Vehicle). If composition were used instead – a single Vehicle class that has-a VehicleType object holding the rate – what would the solution be? Compare the work needed to add a new vehicle type and analyse the time to compute a fee and the space per vehicle object in both designs.
Evidence Expected
A justified inheritance hierarchy, UML diagram, use of super and a traced constructor execution order.

TASK 7 – Override and Specialise Behaviour 4 Marks
Bloom's Level: K4 – Analyse
Identify the behaviour that must differ between subclasses and implement it by method overriding.
Discuss:
• Which methods are overridden (for example: calculateFee()) and why the subclass version is needed
• Use of the @Override annotation and the rules of overriding (signature, return type, access)
• Where the final keyword is appropriate for a variable, method or class, and why
• How toString(), equals() and hashCode() are overridden so that two Vehicle objects with the same registration number are treated as equal

Alternative Concept and Complexity Analysis
Each subclass overrides the fee method. If a single calculateFee() with a switch on a type field were used instead of overriding, what would the solution be? Analyse the time per fee calculation for k vehicle types and the number of classes that must change when a new type is added.
Evidence Expected

Correct overriding with @Override, justified use of final, and correct Object-class method overrides.

TASK 8 – Apply Polymorphism 4 Marks
Bloom's Level: K4 – Analyse
Demonstrate both compile-time and runtime polymorphism in your design.
Discuss:
• At least one overloaded method (for example: calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out)) and how the compiler selects it
• How a single array of Vehicle references can hold different vehicle types (upcasting)
• How dynamic method dispatch selects the correct calculateFee() at runtime
• Why this design avoids long if-else chains that test the vehicle type

Trace one call through dynamic method dispatch, showing the declared type, the actual object type and the method executed.
Complete the Code
Complete the Vehicle hierarchy below and execute FeeDemo. It applies inheritance (Task 6), overriding (Task 7), polymorphism (Task 8) and an interface (Task 9).

```
interface Chargeable {
double chargingFee(int kWh);
}

abstract class Vehicle {
private final String regNo;

protected Vehicle(String regNo) { this.regNo = regNo; }

public String getRegNo() { return regNo; }

protected abstract double ratePerHour();

/** Every started hour is charged in full. */
public double calculateFee(int minutes) {
// TODO 1: Compute billable hours as the ceiling of
```

```
minutes / 60 using integer
// arithmetic only, and return hours × ratePerHour()

}

@Override public boolean equals(Object o) {
return o instanceof Vehicle &&
regNo.equals(((Vehicle) o).regNo);
}

@Override public int hashCode() { return
regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
TwoWheeler(String regNo) { super(regNo); }
@Override protected double ratePerHour() { return
10; }
}

class Car extends Vehicle {
// TODO 2: Write the constructor using super, and
override ratePerHour() to return
// 30

}

class ElectricCar extends Car implements
Chargeable {
ElectricCar(String regNo) { super(regNo); }
// TODO 3: Implement chargingFee() at Rs 12 per
kWh

}

public class FeeDemo {
public static void main(String[] args) {
Vehicle[] parked = { new TwoWheeler("TN01A1"),
new Car("TN02B2"),
new ElectricCar("TN03C3") };
for (Vehicle v : parked) {
double fee = v.calculateFee(130); // dynamic
method dispatch
if (v instanceof Chargeable c) fee +=
```

```
c.chargingFee(10);
System.out.println(v.getClass().getSimpleName() + "
" + v.getRegNo()
+ " -> Rs " + fee);
}
System.out.println(new Car("TN02B2").equals(new
ElectricCar("TN02B2")));
}
}
```

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Explain why the last line prints true, and decide whether this is the correct behaviour for your system.

Alternative Concept and Complexity Analysis

You used dynamic method dispatch. If the same behaviour were written using instanceof checks and casting for each vehicle type, what would the solution be? Analyse the time per call for k vehicle types (O(1) dispatch versus up to k checks) and the effect on code size and maintenance.

Evidence Expected

Working examples of overloading and dynamic dispatch with a traced call and an explanation of the design benefit.

TASK 9 – Introduce Abstraction 4 Marks

Bloom's Level: K4 – Analyse

Use abstract classes and interfaces to separate what the system does from how it is done.

Discuss:

• Which class should be abstract (for example: Vehicle) and which methods should be abstract

• Which capability should be an interface (for example: Chargeable for EVs, Payable for fee-paying users)

• Why you chose an abstract class in one place and

an interface in another
• How a class can obtain multiple behaviours through interfaces
• Where a default or static method in an interface is useful

Alternative Concept and Complexity Analysis
Vehicle is an abstract class and Chargeable is an interface. If Vehicle were changed to an interface with default methods, and Chargeable to an abstract class, what would the solution be? Identify what can no longer be expressed (constructors, state, multiple inheritance) and analyse any difference in time per method call and space per object.
Evidence Expected
A justified use of at least one abstract class and one interface, with the reasoning behind each choice.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers 3 Marks
Bloom's Level: K4 – Analyse
Organise your classes into packages and protect the data of each class.
Discuss:
• Your package structure (for example: parking.model, parking.service, parking.app)
• The access modifier chosen for each field and method (private, default, protected, public) and why
• How getters and setters validate data (for example: rejecting a negative duration or an invalid slot number)
• Where wrapper classes and autoboxing are used, and any risk they introduce (for example: comparing Integer objects with ==)

Alternative Concept and Complexity Analysis
Your fields are private with validating setters. If the fields were made public and validation were done by the calling code, what would the solution be and what could go wrong? Analyse the time cost of validation per update and the number of places

where the same check would have to be repeated.
Evidence Expected
A package structure, justified access modifiers and validated getters and setters that protect the object state.

TASK 11 – Implement, Test and Defend the Design 5 Marks
Bloom's Level: K5 – Evaluate
Implement the complete system in Java and test it. Test and discuss:
• Normal operation: vehicles of different types entering and leaving
• Boundary cases: the last free slot, zero-minute parking, a stay exactly at a fee-slab boundary
• Invalid cases: an unknown vehicle type, a duplicate registration number, an invalid slot number
• A small scenario whose total collection for the day you calculate by hand and compare with the program output
• How well your design follows encapsulation, inheritance, polymorphism and abstraction, and one limitation of it

Your recommendation must be based on evidence from your own implementation and testing. Simply stating that the program runs is not sufficient.
Alternative Concept and Complexity Analysis
Active tickets are currently stored in an array. If they were stored in an ArrayList<ParkingTicket> or a HashMap<String, ParkingTicket> keyed by registration number, what would the exit-processing solution be? Measure and analyse the time to find a ticket on exit and the space used for n parked vehicles in each structure.
Evidence Expected
Working source code, a test table with expected and actual results, and a reasoned evaluation of the design.

DAY-4 DESIGN CHALLENGE
TASK 12 – Re-evaluate and Adapt the Design 8

Marks
Bloom's Level: K5 – Evaluate
Changed Requirement
The university introduces two new vehicle categories – e-bikes (charged a flat fee) and delivery vans (charged at double the car rate during 9 AM – 5 PM). Visitors may now buy a one-day pass. The facility is extended from 3 to 5 floors with a different number of slots on each floor.
Re-evaluate your original design in response to these changed requirements. You may RETAIN → MODIFY → or REDESIGN your original solution.
A. Impact Analysis – 2 Marks
Discuss:
• What has changed from the original problem
• Which of your classes, methods and data structures are affected
• Why a different number of slots on each floor affects your array design
• Which classes, if any, would have to be modified rather than extended

Evidence Expected
A clear analysis of how the changed requirements affect the original design.
B. Evaluation of the Original Design – 2 Marks
Evaluate whether your original design remains appropriate. Discuss:
• Whether new vehicle types can be added only by writing new subclasses
• Whether the pricing rules are spread across many classes or kept in one place
• Whether the existing tested classes can remain unchanged

Evidence Expected
A technically supported evaluation of the original design under the new requirements.
C. Adaptation or Redesign – 2 Marks
Decide whether to retain, modify or redesign the design. If you modify or redesign it:

• Describe the revised class structure (for example: a jagged array for floors, a Pass interface, a PricingRule abstraction)
• Explain what has changed
• Implement the new vehicle categories and the one-day pass

If you retain the original design, provide sufficient technical justification for why no redesign is necessary.
Evidence Expected
An appropriate design response with clear object-oriented reasoning and working code.
D. Evidence-Based Defence – 2 Marks
Defend your final design decision using:
• The number of existing classes that had to change
• The ease of adding a further vehicle type in future
• Test results for the new categories
• Relevant trade-offs

Your final decision must follow logically from your analysis.
Evidence Expected
A well-supported final design decision integrating object-oriented principles, test evidence and the changed requirements.

MARK DISTRIBUTION
Task Bloom's Level Marks
Task 1 – Problem Analysis & Objects K4 – Analyse 4
Task 2 – Data Representation K4 – Analyse 3
Task 3 – Control Logic & Pseudocode K4 – Analyse 4
Task 4 – Classes, Objects & Constructors K4 – Analyse 4
Task 5 – static Members & Object Life Cycle K4 – Analyse 3
Task 6 – Inheritance Hierarchy K4 – Analyse 4
Task 7 – Method Overriding K4 – Analyse 4
Task 8 – Polymorphism K4 – Analyse 4
Task 9 – Abstraction K4 – Analyse 4
Task 10 – Packages & Encapsulation K4 – Analyse 3

Task 11 – Implementation, Testing & Defence K5 – Evaluate 5
Subtotal 42
Task 12 – Day-4 Design Challenge K5 – Evaluate 8
TOTAL 50

Submission Requirements
• GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.
• Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification
Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

UPLOAD PDF FILES * (MAX 2 FILES · 10 MB EACH)

File 1
No file chosen
Choose

File 2 (optional)
No file chosen
Choose

Save Submission

© 2026 **SIMATS Online Lab**. All rights reserved.